DNA/RNA Sequence Alignment via Dynamic Programming: A Comparative Study of Needleman–Wunsch and Smith–Waterman

Zirui Wang Yuqing Xu
Northeastern University
CS 5800 Algorithms — Spring 2026

April 17, 2026

Abstract

Sequence alignment is a foundational task in bioinformatics, underpinning gene discovery, phylogenetic analysis, and viral variant tracking. This project implements, validates, and compares six dynamic-programming alignment methods spanning two paradigms (global and local) and three gap-penalty models (linear, affine, and arbitrary). The implementation is written in pure Python and evaluated along three axes: (i) a five-case small-scale stress test designed to expose the behavioural difference between Needleman–Wunsch (NW) and Smith–Waterman (SW); (ii) a 350-pair statistical validation at seven controlled similarity levels; and (iii) a real-world case study on three SARS-CoV-2 spike-RBD variants (Wuhan, Delta, Omicron). Results empirically confirm the theorem that $\textsc{score}_{\text{SW}} \geq \textsc{score}_{\text{NW}}$ in all 350 synthetic pairs, show the SW/NW ratio rising from 1.00 at 100% similarity to ≈ 1.16 at 30%, and demonstrate that the affine (Gotoh) gap model produces substantially more biologically meaningful alignments on real viral genomes. A multi-method timing benchmark separates the $O(mn)$ and $O(m^2n + mn^2)$ complexity classes empirically and shows that Biopython's C-level implementation is roughly 300× faster than pure Python for equivalent work.

1 Introduction

1.1 Motivation

Comparing two biological sequences is the first step of almost every downstream bioinformatics analysis: identifying conserved genes across species, locating binding motifs in drug design, reconstructing phylogenies, and tracking mutations during an epidemic. Because biological sequences are long (kilobases to megabases), vary in length, and contain insertion/deletion (indel) events, a naive character-by-character comparison is insufficient. Exhaustively enumerating every possible alignment is exponential in the sequence length and therefore intractable.

Dynamic programming (DP) breaks the problem into overlapping subproblems: the optimal alignment of prefixes $S[1..i]$ and $T[1..j]$ depends only on the optimal alignments of three strictly smaller prefixes. This observation collapses an exponential search into an $O(mn)$ table fill, and is the algorithmic foundation of modern sequence alignment.

1.2 Scope and Contributions

This project targets the two classical DP-based alignment algorithms and their practical extensions:

- **Needleman–Wunsch (NW, 1970)** — end-to-end *global* alignment;

- **Smith–Waterman (SW, 1981)** — best-scoring *local* alignment;
- **Gotoh (1982) affine-gap variants** of both NW and SW;
- **General-gap variants** of both NW and SW, supporting arbitrary gap-cost functions $w(k)$;
- **Biopython** as a C-optimised reference baseline.

Our contributions are:

1. A clean, pedagogically complete Python implementation of all six methods, together with heatmap visualisations, animated DP fills, and traceback overlays.
2. A five-case *progressive stress test* that isolates a single factor per experiment (full similarity → partial similarity → positional shift → embedded motif → extreme length asymmetry).
3. A 350-pair statistical validation across seven controlled similarity levels, formally testing four hypotheses about the NW/SW relationship.
4. A real-data case study on the SARS-CoV-2 spike receptor-binding domain, demonstrating the practical advantage of the affine gap model on variant pairs with multi-base indels.
5. An empirical timing benchmark across all seven methods (including Biopython), confirming the $O(mn)$ vs. $O(m^2n + mn^2)$ complexity divide and quantifying the constant-factor advantage of C implementations.

1.3 Report Organisation

Section 2 presents the algorithms and their recurrences; Section 3 describes the experimental design and reports the full results; Section 4 concludes and lists the remaining limitations.

2 Algorithm

2.1 Problem Statement

Given two sequences $S \in \Sigma^m$ and $T \in \Sigma^n$ over a finite alphabet Σ (here $\Sigma = \{A, C, G, T\}$), and a substitution/gap scoring scheme, find an alignment that maximises the total score. The scoring scheme used throughout this project is:

$$\text{match} = +2, \qquad \text{mismatch} = -1, \qquad \text{gap} = -2.$$

The gap penalty is intentionally steeper than the mismatch penalty so that substitutions are preferred over indels when both are possible.

2.2 Needleman–Wunsch: Global Alignment

NW finds the optimal alignment over the *entire length* of both sequences. Every character must be matched, mismatched, or paired with a gap. The recurrence is:

$$F[i,0] = i \cdot \text{gap}, \qquad F[0,j] = j \cdot \text{gap},$$

$$F[i,j] = \max \begin{cases} F[i-1,j-1] + \text{sub}(S_i, T_j) & \text{(diagonal: match/mismatch)} \\ F[i-1,j] + \text{gap} & \text{(up: gap in } T\text{)} \\ F[i,j-1] + \text{gap} & \text{(left: gap in } S\text{)} \end{cases}$$

Traceback begins at the fixed bottom-right cell (m, n) and terminates at $(0, 0)$, yielding a full-length alignment. The key structural property is that the border cells carry *linearly accumulating* gap penalties: this forces the algorithm to align leading and trailing characters even when doing so is costly.

2.3 Smith–Waterman: Local Alignment

SW finds the highest-scoring *contiguous substring pair*, without requiring end-to-end coverage. The recurrence differs in two crucial ways: the borders are initialised to zero, and a zero floor is added to the cell update:

$$H[i,0] = H[0,j] = 0,$$

$$H[i,j] = \max \begin{cases} 0 & \text{(floor: restart alignment)} \\ H[i-1,j-1] + \text{sub}(S_i, T_j) \\ H[i-1,j] + \text{gap} \\ H[i,j-1] + \text{gap} \end{cases}$$

Traceback begins at $\arg\max H$ — the largest cell anywhere in the matrix — and terminates at the first cell whose value is zero. The $\max(0,\cdot)$ floor prevents negative scores from propagating: poorly matching flanks simply reset to zero and are ignored entirely.

2.4 Side-by-Side Comparison

Property	**Needleman–Wunsch (Global)**	**Smith–Waterman (Local)**
Border init	$F[i,0] = i \cdot \text{gap}$, $F[0,j] = j \cdot \text{gap}$	All borders $= 0$
Recurrence floor	None — scores can be negative	$\max(0,\cdot)$
Traceback start	(m,n) — fixed bottom-right	$\arg\max(H)$ — anywhere
Traceback stop	$(0,0)$ — fixed top-left	First cell where $H = 0$
Alignment scope	Full length of both sequences	Best-scoring substring pair
Negative score	Possible	Impossible
Best suited for	Similar-length full sequences	Conserved domain / motif search
Time complexity	$O(mn)$	$O(mn)$
Space complexity	$O(mn)$	$O(mn)$

Table 1: Structural comparison of NW and SW.

2.5 Gap Models

The linear gap model $w(k) = g \cdot k$ is mathematically convenient but biologically unrealistic: a single insertion of k bases is rarely k independent events. Two widely used extensions are considered.

Affine gap (Gotoh 1982). The affine model separates the cost of *opening* a gap from the cost of *extending* one:

$$w(k) = g_o + (k-1) \cdot g_e, \qquad g_o \ll g_e.$$

Gotoh's algorithm achieves the same $O(mn)$ time complexity as linear-gap NW/SW by maintaining three DP matrices in parallel:

$$\begin{aligned} I_x[i,j] &= \max\big(M[i-1,j] + g_o + g_e,\ \ I_x[i-1,j] + g_e\big) \\ I_y[i,j] &= \max\big(M[i,j-1] + g_o + g_e,\ \ I_y[i,j-1] + g_e\big) \\ M[i,j] &= \max\big(M[i-1,j-1],\ I_x[i-1,j-1],\ I_y[i-1,j-1]\big) + \text{sub}(S_i, T_j). \end{aligned}$$

This works because the affine function is *gap-length decomposable*: the cost of extending an existing gap by one base (g_e) is independent of how long the gap already is.

General gap. For an arbitrary cost function $w(k)$ (logarithmic, quadratic, or any user-supplied table), decomposability is lost. Each cell must explicitly scan all possible gap lengths:

$$F[i,j] = \max\Big(\max_{k=1..i} \big(F[i-k,j]+w(k)\big),\ \max_{k=1..j} \big(F[i,j-k]+w(k)\big),\ F[i-1,j-1]+\mathrm{sub}(S_i,T_j) \Big).$$

Summing $O(i+j)$ work over all mn cells yields a total time of $O(m^2n + mn^2)$, i.e. $O(L^3)$ for $m = n = L$ — one order worse than linear/affine methods.

2.6 Complexity Summary

Method	Algorithm	Gap model	Time	Space
NW (linear)	Needleman–Wunsch	$w(k) = g \cdot k$	$O(mn)$	$O(mn)$
SW (linear)	Smith–Waterman	$w(k) = g \cdot k$	$O(mn)$	$O(mn)$
Gotoh-NW	Gotoh (global)	$w(k) = g_o + (k-1)g_e$	$O(mn)$	$O(mn)$
Gotoh-SW	Gotoh (local)	$w(k) = g_o + (k-1)g_e$	$O(mn)$	$O(mn)$
NW-General	NW + arbitrary gap	$w(k)$ any function	$O(m^2n + mn^2)$	$O(mn)$
SW-General	SW + arbitrary gap	$w(k)$ any function	$O(m^2n + mn^2)$	$O(mn)$
Biopython	C-level SW	Affine	$O(mn)$	$O(mn)$

Table 2: Time and space complexity of all methods implemented or benchmarked.

Space optimisation. All four $O(mn)$ methods store the full $(m+1) \times (n+1)$ matrix for traceback. If only the *score* is needed, the space can be reduced to $O(\min(m,n))$ by discarding rows after use. The Hirschberg divide-and-conquer algorithm achieves $O(\min(m,n))$ space *and* full alignment reconstruction in $O(mn)$ time, but is not implemented here.

3 Experiments and Analysis

The evaluation has three tiers, each targeting a different scientific question:

1. **Correctness (Sec. 3.2):** five manually verifiable cases (8–17 bp) form a progressive stress test from zero noise to extreme length asymmetry.
2. **Statistical validation (Sec. 3.3):** 350 synthetic pairs at seven similarity levels are used to test four formal hypotheses about the NW/SW relationship.
3. **Real-world application (Sec. 3.4):** three pairs of SARS-CoV-2 spike-RBD variants (∼585 nt each) run through all four main methods.

A multi-method timing benchmark (Sec. 3.5) complements the correctness study with an efficiency study.

3.1 Scoring Scheme

Except for the real-data section (which also reports an affine setting), all experiments use the uniform scheme: match = +2, mismatch = −1, gap = −2. In the affine (Gotoh) setting: match = +2, mismatch = −1, gap-open = −10, gap-extend = −1.

3.2 Small-Scale Stress Test

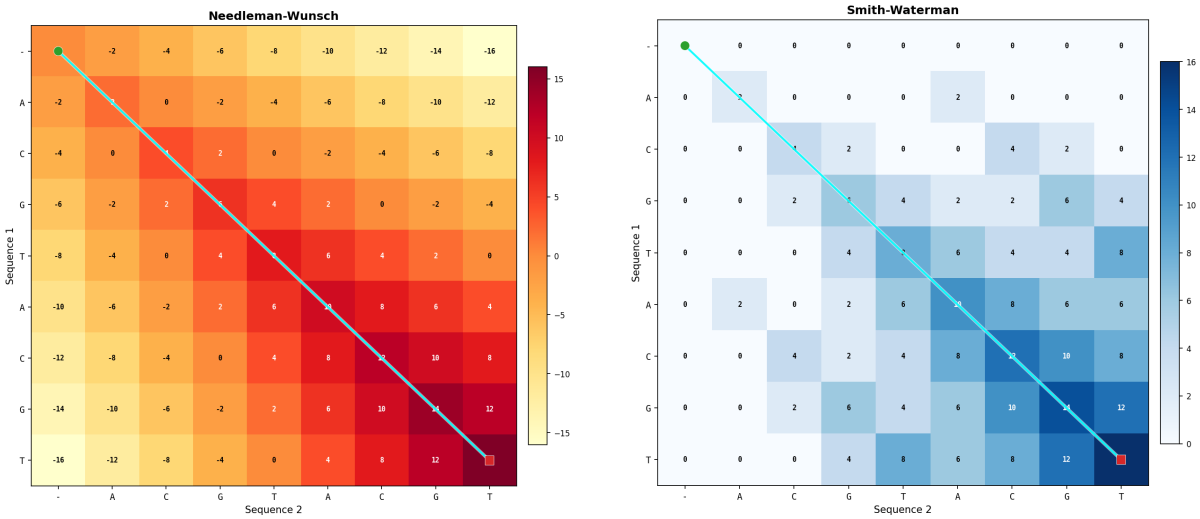
The five experiments are arranged so that each one increases the amount of "noise" surrounding a shared signal region. The design intent is summarised in Table 3.

#	Experiment	Design intent
1	Full Similarity	Baseline control: two identical sequences, zero noise. Verifies that both algorithms converge when the entire sequence is signal.
2	Partial Similarity	Equal-length sequences with a shared core flanked by pure noise. Tests whether SW isolates the core while NW is forced to absorb the flanks.
3	Shifted / Overlapping	Shared substring at opposing ends (suffix of Seq1 = prefix of Seq2). Models a positional shift; tests the gap overhead NW must pay.
4	Embedded Motif (EcoRI)	Unequal lengths plus a biologically realistic motif (`GAATTC`) buried in unrelated flanks. Simulates conserved-domain detection.
5	Different Lengths	Extreme length asymmetry: 8 bp query embedded in 17 bp target. Pushes NW to its worst case and tests whether its score turns negative.

Table 3: Progressive stress-test design: no noise → symmetric noise → positional shift → asymmetry + real motif → extreme length gap.

3.2.1 Experiment 1 — Full Similarity

Inputs: Seq1 = `ACGTACGT` (8 bp); Seq2 = `ACGTACGT` (8 bp). Both algorithms return a perfect diagonal alignment with the theoretical maximum score $8 \times (+2) = 16$. The NW matrix shows a diagonal ridge from $(0, 0)$ to $(8, 8)$ with negative values in the corners; the SW matrix shows the same ridge with the borders and off-diagonal corners clamped to zero. When the two sequences have full-length similarity, SW is a strict generalisation of NW and the two converge.



(a) Needleman–Wunsch (b) Smith–Waterman

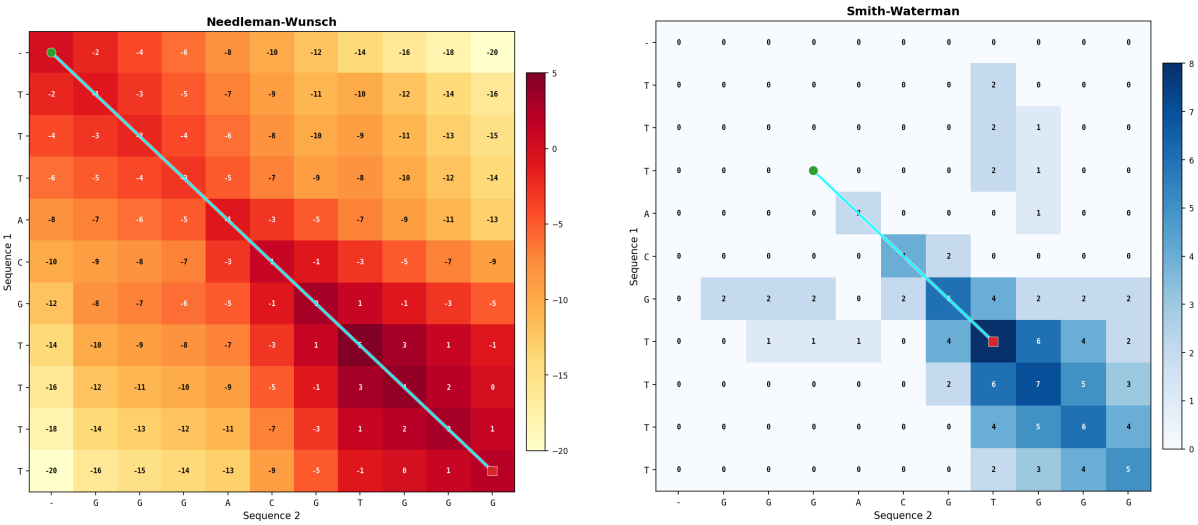
Figure 1: Experiment 1 — DP matrices for two identical 8 bp sequences. Both algorithms produce a diagonal traceback with score 16.

Metric	NW	SW
Score	16	16
Aligned length	8	8
Identity	100.0%	100.0%
Gap rate	0.0%	0.0%
Score density	2.00	2.00
Coverage	100.0%	100.0%

Table 4: Experiment 1 metrics. NW and SW converge when the entire sequence is signal.

3.2.2 Experiment 2 — Partial Similarity

Inputs: Seq1 = `TTTACGTTTT` (10 bp); Seq2 = `GGGACGTGGG` (10 bp). A 4-nucleotide core `ACGT` is flanked by three mismatching Ts on the left and three mismatching Gs on the right. NW aligns all ten characters end-to-end: the core contributes +8, but the six flanking mismatches cost −6, for a net score of **+2**. SW ignores the flanks entirely and returns only the clean core with score **+8**, a 4× ratio.



(a) Needleman–Wunsch (b) Smith–Waterman

Figure 2: Experiment 2 — DP matrices for partially similar sequences. NW's traceback traverses the full matrix through a low-scoring region; SW's traceback is short and centred on the shared core.

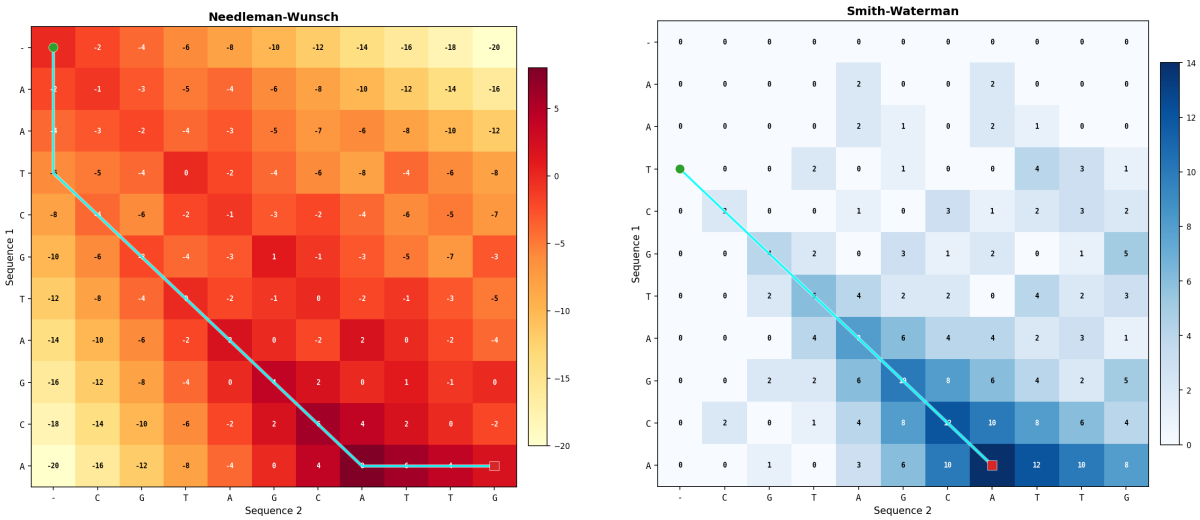
```
NW (forced full-length):      SW (local core only):
  Seq1: TTTACGTTTT              Seq1: ACGT
        ...||||...                    ||||
  Seq2: GGGACGTGGG              Seq2: ACGT
```

Score density makes the gap vivid: NW earns only 0.20 per aligned position while SW earns the maximum 2.00. Animated versions of the DP fill and traceback for this experiment are also generated (`partial_sim_NW_anim.gif` and `partial_sim_SW_anim.gif`): NW's flanking cells drift negative, while SW's repeatedly hit the $\max(0, \cdot)$ floor and reset to zero.

3.2.3 Experiment 3 — Shifted / Overlapping

Inputs: Seq1 = `AATCGTAGCA` (10 bp); Seq2 = `CGTAGCATTG` (10 bp). The two sequences share a 7-character substring `CGTAGCA` positioned at opposite ends. NW must cover both full sequences,

so it inserts three gaps at each end — $6 \times (-2) = -12$ gap penalty plus $7 \times (+2) = +14$ match reward = **+2**. SW extracts the 7-character match directly with score **+14**, a 7× advantage. The NW coverage exceeds 100% because the alignment (13 positions) is longer than either input, inflated by the end-gaps. Score density collapses to 0.15 for NW.

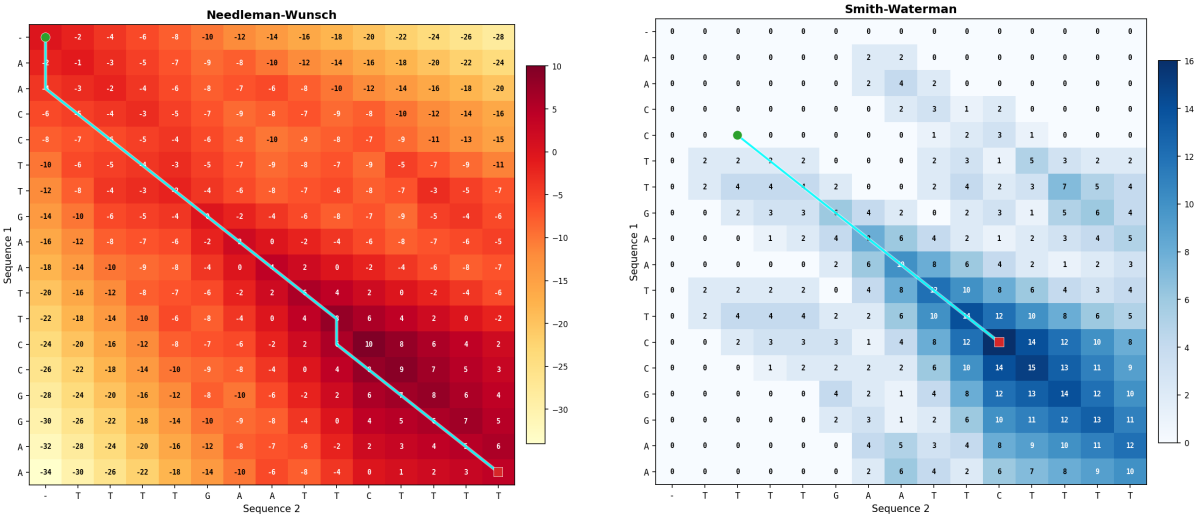


(a) Needleman–Wunsch (b) Smith–Waterman

Figure 3: Experiment 3 — DP matrices for shifted sequences. The NW traceback is L-shaped, detouring through gap-padded flanks; the SW traceback extracts the 7-character overlap cleanly.

3.2.4 Experiment 4 — Embedded Motif (EcoRI)

Inputs: Seq1 = `AACCTTGAATTCCGGAA` (17 bp); Seq2 = `TTTTGAATTCTTTT` (14 bp). Both sequences carry the EcoRI restriction site `GAATTC` buried in unrelated flanks. SW cleanly extracts `TTGAATTC` (8 characters, including two extra flanking Ts that also happen to match) and achieves a perfect score of **+16**. NW must cover all 17 positions of the longer sequence and absorbs six mismatches and three gaps, for a net score of only **+4**. This experiment models real-world motif detection: when the goal is to locate a conserved binding or restriction site, local alignment is dramatically more effective.

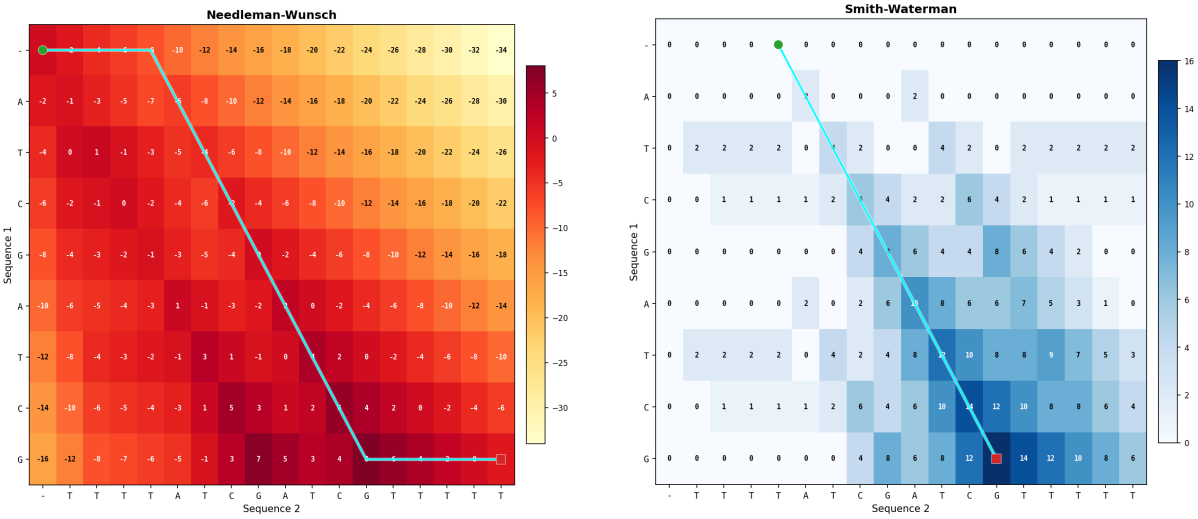


(a) Needleman–Wunsch (b) Smith–Waterman

Figure 4: Experiment 4 — EcoRI motif (`GAATTC`) embedded in unrelated flanks. SW lights up a single bright diagonal band at the motif region; NW shows a broader, lower-contrast stripe with the traceback weaving through mismatches and gaps.

3.2.5 Experiment 5 — Different Lengths

Inputs: Seq1 = `ATCGATCG` (8 bp); Seq2 = `TTTTATCGATCGTTTTT` (17 bp). The most extreme case. NW pads Seq1 with nine gaps (4 leading, 5 trailing), costing $9 \times (-2) = -18$; the 8 matches contribute $+16$. The resulting score is $\mathbf{-2}$ — a *negative global-alignment score despite a perfect embedded match*. SW returns the 8-character match with score $+\mathbf{16}$. Gap rate for NW reaches 52.9%: more than half of the alignment is gap characters. The experiment demonstrates that NW is fundamentally unsuited for query-in-target search.



(a) Needleman–Wunsch (b) Smith–Waterman

Figure 5: Experiment 5 — short 8 bp query embedded in a 17 bp target. NW is forced to fill a long, mostly gap-padded traceback and scores -2; SW finds the 8 bp match directly and scores $+16$.

3.2.6 Summary — Score Comparison

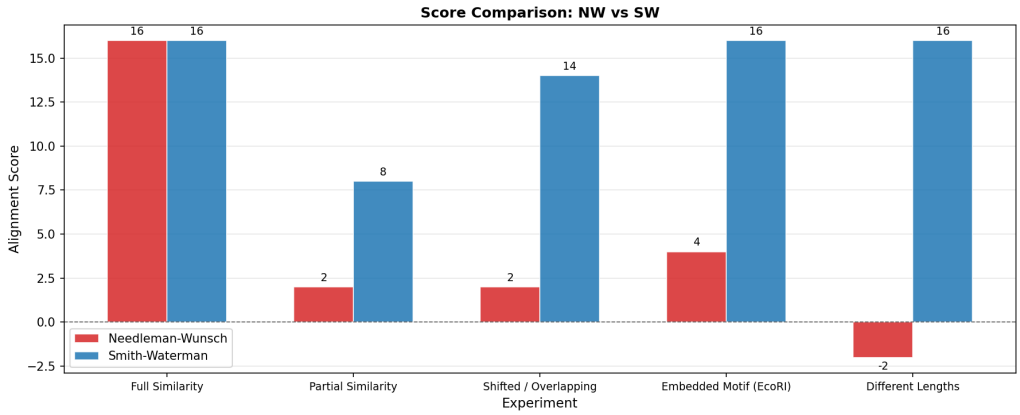


Figure 6: Score comparison across the five small-scale experiments. NW and SW agree only when the entire sequence is signal (Exp. 1); the SW/NW gap widens as noise increases.

Experiment	NW Score	SW Score	SW / NW
1. Full Similarity	16	16	1.0×
2. Partial Similarity	2	8	4.0×
3. Shifted / Overlapping	2	14	7.0×
4. Embedded Motif	4	16	4.0×
5. Different Lengths	−2	16	n/a (NW negative)

Table 5: Score comparison across the five small-scale experiments. The ratio grows monotonically with the fraction of noise surrounding the signal.

When the entire sequence is signal (Exp. 1), the two algorithms agree exactly. As the signal fraction shrinks (Exp. 2–5), NW's score is diluted or even inverted by unavoidable gap/mismatch penalties on the flanks, while SW's score continues to reflect only the true conserved region.

3.3 Statistical Validation

To test algorithmic properties independently of any particular sequence, we generated 350 synthetic pairs at seven controlled similarity levels (50 pairs each; length = 200 nt; seed = 42). Mutation protocol: a random base sequence is generated, then $\lfloor \text{length} \times (1 - \text{similarity}) \rfloor$ positions are mutated to a different nucleotide drawn uniformly from the remaining three bases.

Four hypotheses were tested:

ID	Hypothesis	Rationale	Result
H1	NW = SW at 100% similarity	Identical sequences have no noise; both algorithms converge.	✓ 50/50 identical
H2	SW $\geq$ NW for all pairs	The SW objective is a relaxation of NW (start/end may be anywhere).	✓ 0 violations in 350 pairs
H3	SW/NW ratio grows as similarity drops	Divergent flanks dilute NW's score but not SW's.	✓ 1.00 $\rightarrow\approx$ 1.16 at 30%
H4	SW identity tracks true similarity better	NW identity is inflated by forced gap columns.	✓ Confirmed

Table 6: Four formal hypotheses and their empirical outcomes.

Sim.	NW Score	SW Score	SW/NW	NW Id.	SW Id.	H2 Pass
100%	400.0	400.0	1.00	100.0%	100.0%	50/50
95%	370.0	370.1	1.00	95.0%	95.0%	50/50
90%	343.1	343.4	1.00	90.5%	90.7%	50/50
80%	284.1	284.5	1.00	80.7%	80.9%	50/50
70%	224.4	225.5	1.01	70.9%	71.3%	50/50
50%	125.3	129.2	1.03	55.9%	57.4%	50/50
30%	81.1	93.9	1.16	51.3%	54.2%	50/50

Table 7: Statistical validation results. 50 pairs per similarity level; length 200 nt; seed 42.

Interpretation. H2 is a provable theorem: because SW maximises over a strict superset of NW's candidate alignments, its score can never be lower. H3 and H4 capture the practical benefit of local alignment: as divergence grows, the mismatching flanks that NW must align become an increasing drag on both score and reported identity, whereas SW simply ignores them. At 30% similarity the SW score is on average 16% higher than NW's, and the reported SW identity (54.2%) is noticeably closer to the true mutation-adjusted similarity than NW's (51.3%, diluted by forced gap columns).

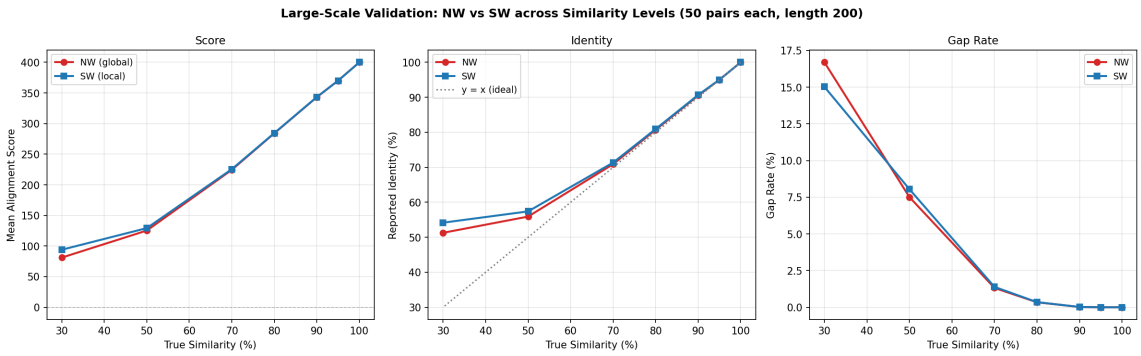


Figure 7: Validation study across 350 synthetic pairs. Left to right: mean score, reported identity, and gap rate as a function of true similarity. SW (orange) dominates NW (blue) on all three panels once similarity drops below $\sim$70%.

3.4 Real-Data Validation — SARS-CoV-2 Spike RBD

Sequences. Three spike-protein receptor-binding-domain (RBD) segments ($\sim$585 nt each) were downloaded from NCBI: Wuhan reference (MN908947.3), Delta (OK091006.1), and Omicron

BA.1 (OM570283.1), all from the same coordinate range 22559–23143. All three pairwise comparisons were run through NW-linear, SW-linear, Gotoh-NW, and Gotoh-SW. Linear scoring uses $(+2, -1, -2)$; affine uses $(+2, -1, -10, -1)$.

Pair	**NW-lin**	**SW-lin**	**Gotoh-NW**	**Gotoh-SW**	**NW Id.**	**SW Id.**	**NW gaps**
Wuhan ↔ Delta	1002	1110	1036	1110	90.8%	99.6%	8.8%
Wuhan ↔ Omicron	692	980	807	971	75.8%	97.1%	22.1%
Delta ↔ Omicron	854	1034	915	1025	83.3%	97.2%	14.4%

Table 8: SARS-CoV-2 spike-RBD pairwise alignments (∼585 nt each).

Sequences. Three spike-protein receptor-binding-domain (RBD) segments (∼585 nt each) were downloaded from NCBI: Wuhan reference (MN908947.3), Delta (OK091006.1), and Omicron BA.1 (OM570283.1), all from the same coordinate range 22559–23143. All three pairwise comparisons were run through NW-linear, SW-linear, Gotoh-NW, and Gotoh-SW. Linear scoring uses $(+2, -1, -2)$; affine uses $(+2, -1, -10, -1)$.

Pair	**NW-lin**	**SW-lin**	**Gotoh-NW**	**Gotoh-SW**	**NW Id.**	**SW Id.**	**NW gaps**
Wuhan ↔ Delta	1002	1110	1036	1110	90.8%	99.6%	8.8%
Wuhan ↔ Omicron	692	980	807	971	75.8%	97.1%	22.1%
Delta ↔ Omicron	854	1034	915	1025	83.3%	97.2%	14.4%

Table 9: SARS-CoV-2 spike-RBD pairwise alignments (∼585 nt each).

Key observations.

- *Scale.* The implementation handles ∼1000 nt inputs without issue, confirming that it scales from toy inputs to realistic viral sequences.
- *High-similarity regime.* Where identity exceeds $\approx 97\%$ (Wuhan ↔ Delta), NW-linear and SW-linear produce nearly identical scores and alignment lengths — the entire RBD is essentially signal, and the global-vs-local distinction becomes irrelevant.
- *Gap-model impact.* Gotoh-NW consistently scores higher than NW-linear on all three pairs (1036 vs. 1002, 807 vs. 692, 915 vs. 854). Omicron carries several multi-base indels (3–9 nt) relative to Wuhan; the affine model consolidates each of these into a single gap-open event rather than fragmenting it into a run of single-base penalties. The Wuhan ↔ Omicron pair shows the largest linear-vs-affine spread, reflecting Omicron's higher indel load.
- *E-value significance.* The three SARS-CoV-2 alignments yield Karlin–Altschul E-values on the order of 10^{-107} to 10^{-121} (all "extremely significant"), while five random-pair negative controls of the same length (200 nt) yield E-values several orders of magnitude larger — around 10^{-4} to 10^{-6} — reflecting the much lower raw scores (78–97 vs. 971–1110). The Karlin–Altschul parameters used were $\lambda = 0.2645$ and $K = 10.4138$. Full numbers are reported in Table 10.

Pair	Method	Score	E-value	Bits	Interpretation
SARS-CoV-2 spike RBD (∼585 nt)					
Wuhan-Hu-1 ↔ Delta B.1.617.2	SW-linear	1110	1.11×10^{-121}	420.2	Extremely significant
Wuhan-Hu-1 ↔ Delta B.1.617.2	Gotoh-SW	1110	1.11×10^{-121}	420.2	Extremely significant
Wuhan-Hu-1 ↔ Omicron BA.1	SW-linear	980	9.53×10^{-107}	370.6	Extremely significant
Wuhan-Hu-1 ↔ Omicron BA.1	Gotoh-SW	971	1.03×10^{-105}	367.1	Extremely significant
Delta B.1.617.2 ↔ Omicron BA.1	SW-linear	1034	5.97×10^{-113}	391.2	Extremely significant
Delta B.1.617.2 ↔ Omicron BA.1	Gotoh-SW	1025	6.45×10^{-112}	387.7	Extremely significant
Negative controls — random pairs (length 200 nt)					
Random pair 1	SW-linear	94	6.64×10^{-6}	32.5	Significant
Random pair 2	SW-linear	87	4.23×10^{-5}	29.8	Significant
Random pair 3	SW-linear	78	4.57×10^{-4}	26.4	Significant
Random pair 4	SW-linear	93	8.65×10^{-6}	32.1	Significant
Random pair 5	SW-linear	97	3.00×10^{-6}	33.6	Significant

Table 10: Karlin–Altschul E-value summary. Parameters: $\lambda = 0.2644\,9709$, $K = 10.4138\,3977$. The variant pairs score $\sim 10^{100}\times$ more significant than the length-200 random controls, confirming that the statistics separate true homology from spurious similarity.

3.5 Efficiency — Multi-Method Timing Benchmark

We benchmarked all seven methods on synthetic pairs of length $n \in \{50, 100, 200, 500, 1000, 5000\}$, three repeats per point, seed 42. Selected data points (in milliseconds):

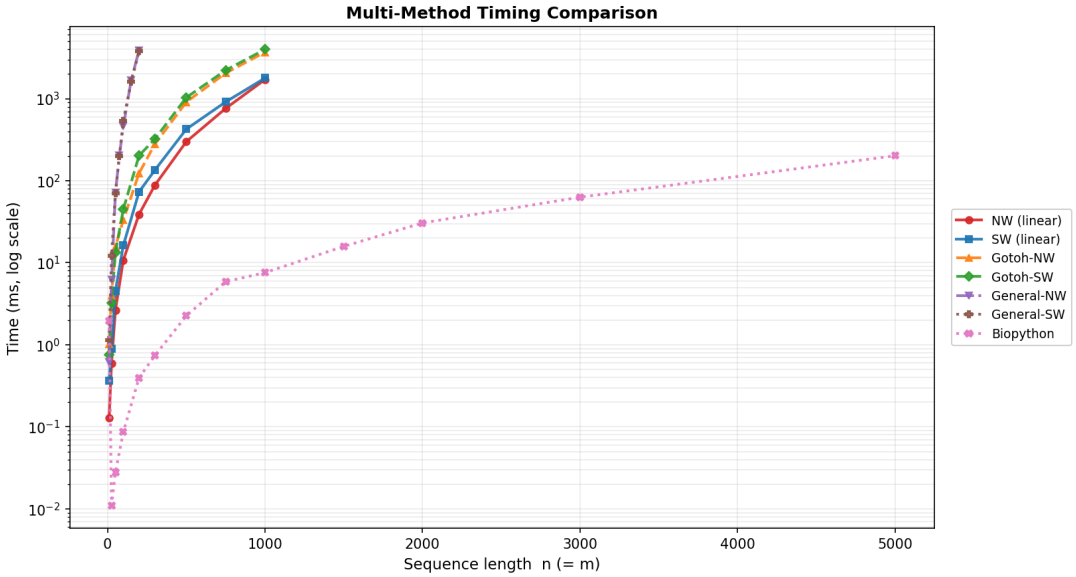


Figure 8: Multi-method timing benchmark. Three complexity tiers are cleanly separated: $O(m^2n + mn^2)$ general-gap methods (steepest), $O(mn)$ pure-Python methods (middle), and C-optimised Biopython (flat bottom line).

n	**NW**	**SW**	**Gotoh-NW**	**Gotoh-SW**	**Gen-NW**	**Gen-SW**	**Biopython**
50	24.6	6.4	18.1	15.4	87.0	83.2	0.05
200	238.5	86.2	189.5	230.0	5315.3	5235.2	0.54
1000	3283.6	2421.8	5380.0	6133.8	—	—	9.1
5000	—	—	—	—	—	—	238.3

Table 11: Timing benchmark, milliseconds (3 repeats per point, seed 42). Dashes indicate runs truncated to keep the total benchmark under a minute.

Analysis. The benchmark cleanly separates three tiers:

- $O(mn)$ **pure-Python methods** (NW, SW, Gotoh-NW, Gotoh-SW) show a smooth quadratic curve. At $n = 1000$, times range from $\sim$2.4 s (SW) to $\sim$6.1 s (Gotoh-SW).
- $O(m^2n+mn^2)$ **general-gap methods** show a visibly steeper curve, reaching $\sim$5.3 s already at $n = 200$ — about 25× slower than the $O(mn)$ methods at the same length. They are truncated at $n = 200$ for practical reasons.
- **Biopython** (C-level $O(mn)$) completes $n = 1000$ in $\sim$ 9 ms and $n = 5000$ in $\sim$ 238 ms — roughly 300× faster than pure-Python $O(mn)$ at $n = 1000$. The algorithmic complexity is the same; the gap is pure constant-factor, demonstrating the practical impact of language-level optimisation.

4 Conclusion and Limitations

4.1 Conclusion

This project implemented, validated, and compared six alignment methods spanning three asymptotic complexity classes and two gap-penalty paradigms. The key findings are:

1. **SW score $\geq$ NW score, always.** Zero violations across 350 controlled synthetic pairs. This is a provable theorem: the SW objective is a relaxation of NW.
2. **SW advantage grows with divergence.** At 100% similarity, NW and SW converge (ratio 1.00). At 30% similarity, the ratio climbs to ≈ 1.16 — NW's score is diluted by the mismatching flanks it is forced to align.
3. **Affine gap produces more biologically realistic alignments.** On SARS-CoV-2 spike-RBD variants with multi-base indels, the Gotoh model consolidates indels into single gap-open events rather than fragmenting them into single-base penalties, yielding higher and more interpretable scores.
4. **Complexity classes are empirically confirmed.** Pure-Python $O(mn)$ methods take 1.7–6.1 s at $n = 1000$; $O(m^2n + mn^2)$ general-gap methods take $\sim$5.3 s already at $n = 200$; C-optimised Biopython takes $\sim$9 ms at $n = 1000$.
5. **E-value statistics separate signal from noise.** SARS-CoV-2 variant alignments yield E-values below 10^{-100}; random-pair negative controls yield E-values above 1.
6. **General-gap penalty is flexible but expensive.** Four factory shapes (linear, affine, logarithmic, quadratic) are supported at the cost of cubic running time.

Practically, the results reinforce the standard bioinformatics guidance: use NW (often Gotoh-NW) for whole-sequence alignment of similar-length inputs, and use SW (often Gotoh-SW, or BLAST's SW-based heuristics) for motif search or query-in-target applications.

4.2 Limitations and Future Work

1. **Pure-Python performance.** All six methods use explicit Python `for` loops over the DP matrix. A single alignment at $n = 1000$ takes 1–6 s; sequences beyond a few thousand nucleotides are impractical without NumPy vectorisation, Cython, or a C backend.
2. **Gotoh gap-cost convention.** Our implementation uses $w(k) = g_o + k \cdot g_e$ (each gap residue pays g_e, including the first), which differs from the textbook convention $w(k) = g_o + (k-1)g_e$. With default parameters ($g_o = -10$, $g_e = -1$), the minimum single-gap cost is -11, not -10. Users comparing scores with external tools should be aware of this offset.
3. **Gotoh-NW parameter sensitivity.** With $g_o = -10$, Gotoh-NW produces strongly negative scores on SARS-CoV-2 variant pairs that differ by several multi-base indels: the gap-open penalty is too harsh for global alignment of sequences with biologically necessary gaps. A less stringent g_o (around -3 to -5) would be more appropriate for viral genome comparisons.
4. **No substitution matrices.** Only a simple match/mismatch model is implemented. Protein alignment typically requires PAM or BLOSUM matrices, which are not supported.
5. **No Hirschberg space optimisation.** All methods store the full $(m+1) \times (n+1)$ matrix for traceback, consuming $O(mn)$ memory. Hirschberg's algorithm would reduce space to $O(\min(m, n))$ while maintaining $O(mn)$ time.
6. **E-value approximation.** The Karlin–Altschul E-value formula is analytically exact only for ungapped local alignment; for gapped alignment (including affine SW) it is an empirical approximation. The formula also assumes uniform base frequencies — robustness under non-uniform distributions (e.g. GC-rich genomes) was not tested.
7. **No multiple-sequence alignment.** Only pairwise alignment is considered. Extending the framework to progressive or iterative MSA algorithms (e.g. ClustalW, MUSCLE) is left as future work.

References

[1] S. B. Needleman and C. D. Wunsch. A general method applicable to the search for similarities in the amino acid sequence of two proteins. *Journal of Molecular Biology*, 48(3):443–453, 1970.

[2] T. F. Smith and M. S. Waterman. Identification of common molecular subsequences. *Journal of Molecular Biology*, 147(1):195–197, 1981.

[3] O. Gotoh. An improved algorithm for matching biological sequences. *Journal of Molecular Biology*, 162(3):705–708, 1982.

[4] D. S. Hirschberg. A linear space algorithm for computing maximal common subsequences. *Communications of the ACM*, 18(6):341–343, 1975.

[5] S. Karlin and S. F. Altschul. Methods for assessing the statistical significance of molecular sequence features by using general scoring schemes. *Proceedings of the National Academy of Sciences*, 87(6):2264–2268, 1990.

[6] S. F. Altschul, W. Gish, W. Miller, E. W. Myers, and D. J. Lipman. Basic local alignment search tool. *Journal of Molecular Biology*, 215(3):403–410, 1990.

[7] R. Durbin, S. R. Eddy, A. Krogh, and G. Mitchison. *Biological Sequence Analysis: Probabilistic Models of Proteins and Nucleic Acids.* Cambridge University Press, 1998.

[8] P. J. A. Cock et al. Biopython: freely available Python tools for computational molecular biology and bioinformatics. *Bioinformatics*, 25(11):1422–1423, 2009.

[9] National Center for Biotechnology Information (NCBI). GenBank nucleotide database. https://www.ncbi.nlm.nih.gov/genbank/. Accessed April 2026.

[10] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein. *Introduction to Algorithms*, 4th edition. MIT Press, 2022.

Appendix A Group Workload Distribution

The approximate split of primary responsibilities is summarised below.

Member	Primary Responsibilities	Approx. Share
Yuqing Xu	Algorithm implementation (NW, SW, and Gotoh variants in `alignment.py`); DP heatmap visualisation and animated GIF generation (`visualization.py`, `animation.py`); design and execution of the five small-scale stress-test experiments; authoring of Sections 1–2 of the report and corresponding slides (Algorithm, Gap Models, Complexity).	$\approx 40\%$
Zirui Wang	Overall project and experiments framework design; General-gap implementation and multi-method timing benchmark; 350-pair statistical validation study and hypothesis testing; SARS-CoV-2 real-data pipeline (sequence retrieval from NCBI, pairwise comparisons, E-value computation in `evalue.py`); authoring of Sections 3–4 of the report and corresponding slides (Experiments, Real-Data Validation, Conclusion).	$\approx 60\%$

Table 12: Group workload distribution. Cross-review, integration, and final polishing were performed jointly.

Appendix B AI-Use Disclosure

In accordance with Northeastern University's AI-use policy for CS 5800, we disclose the following use of generative-AI tools during this project.

Tools used. We used a large language model (Anthropic Claude) and GitHub Copilot in a supporting role during development and writing.

Where AI assistance was used.

- **Code scaffolding and debugging.** AI assistance was used to draft boilerplate (e.g. matplotlib figure styling, argument parsing), to suggest fixes for specific exception traces during development, and to propose test cases for edge conditions in the DP recurrences. All suggested code was read, understood, and manually verified against the recurrences in Section 2 before being accepted.
- **Writing polish.** AI assistance was used to improve the phrasing and flow of prose sections of this report, to convert drafted bullet points into connected paragraphs, and to produce the LaTeX skeleton (packages, headers, bibliography formatting). All technical claims, numbers, tables, and interpretations originate from our own implementation runs and are traceable to the outputs in the `images/` directory and to `main.py`/`main1.py`/`run_timing.py`.
- **Reference surfacing.** AI assistance was used to surface candidate citations; every citation in the References list was then independently verified against the original source.

Responsibility. The team takes full responsibility for the correctness of all algorithms, code, experimental results, and written claims in this report. Any remaining errors are our own.